INFORME DE PRÁCTICAS EN EMPRESA

1. Introducción

El presente informe recoge las actividades realizadas durante el período de prácticas en empresa en el marco del módulo Introducción a la Nube Pública. En concreto, se documenta el desarrollo de un proyecto web real desplegado en la nube: una aplicación de quiniela de empresa denominada PorraIDS, construida con Node.js, con base de datos PostgreSQL gestionada en un servicio cloud externo y desplegada en una plataforma PaaS.

Este proyecto ha permitido aplicar de manera práctica los conceptos clave del módulo: comparativa y selección de proveedores cloud, gestión de bases de datos como servicio, despliegue en plataformas PaaS, seguridad en el acceso, y monitorización del consumo y costes.

2. Resultados de Aprendizaje y Criterios de Evaluación

RA01 — Fundamentos de la computación en la nube, ventajas y gestión de costes

CE e) Uso correcto de herramientas para estimar y gestionar presupuestos

Durante el proyecto se realizó una comparativa real entre distintos proveedores de bases de datos cloud con el objetivo de seleccionar la opción más adecuada en términos de coste, funcionalidad y sostenibilidad a largo plazo:

- Google Cloud SQL (Cloud SQL for PostgreSQL): . Se identificó que ofrece $300 de crédito inicial gratuito válido durante 90 días, tras los cuales comienza la facturación. Se descartó por la ausencia de tier gratuito permanente.
- Render PostgreSQL: plataforma PaaS con plan gratuito que incluye una base de datos PostgreSQL. Se detectó como limitación crítica que el plan gratuito expira a los 90 días, lo que obliga a recrear la base de datos periódicamente y actualizar las variables de entorno. Se descartó por falta de persistencia garantizada.
- Neon (neon.tech): servicio PostgreSQL serverless con plan gratuito permanente, 512 MB de almacenamiento, sin necesidad de tarjeta de crédito y sin caducidad. Se seleccionó como solución definitiva por su modelo de costes predecible y su adecuación al caso de uso.
- Supabase: alternativa PostgreSQL gratuita permanente con 500 MB. Se evaluó como opción equivalente a Neon.

Proveedor	Coste	Caduca	Decisión
Google Cloud SQL	Sin plan gratuito	90 días (créditos)	Descartado
Render PostgreSQL	Gratis	Sí, 90 días	Descartado
Neon	Gratis permanente	No	Seleccionado
Supabase	Gratis permanente	No	Alternativa viable

CE f) Participación en actividades prácticas sobre gestión de costes

Tras poner en producción la aplicación en Neon, se realizó un seguimiento activo del consumo real de recursos dentro del dashboard de Neon. Se observó y analizó:

- Compute: 0,24 / 100 CU-hrs consumidos en el período inicial. Se comprendió que el modelo de Neon es serverless: la base de datos entra en modo sleep cuando no hay actividad, lo que reduce el consumo al mínimo.
- Storage: 0,03 / 0,5 GB utilizados. Se concluyó que el volumen de datos generado por la aplicación (apuestas, jugadores, semanas) es mínimo y el plan gratuito es más que suficiente.
- Network transfer: prácticamente 0, dado que las peticiones son JSON de tamaño reducido.
- Se verificó que el límite de compute se resetea mensualmente, lo que garantiza la sostenibilidad del plan gratuito a largo plazo.

RA02 — Infraestructura global, seguridad y gestión de accesos

CE d) Comprensión del modelo de responsabilidad compartida en la nube

En el proyecto se identificaron claramente las responsabilidades de cada parte dentro del modelo de responsabilidad compartida:

- Responsabilidad del proveedor cloud (Neon / Render): disponibilidad de la infraestructura, mantenimiento de los servidores físicos, actualizaciones del motor PostgreSQL, cifrado del almacenamiento en reposo y backups automáticos.
- Responsabilidad del desarrollador (mía): seguridad de la aplicación, gestión de credenciales, control de acceso mediante autenticación, no exposición de datos sensibles en el código fuente, y correcta configuración de las variables de entorno.

CE e) Aplicación de medidas de seguridad básicas mediante herramientas de gestión de acceso

Se implementaron diversas medidas de seguridad en la aplicación:

- Autenticación mediante express-session: sistema de login con credenciales de administrador gestionadas como variables de entorno (ADMIN_USER, ADMIN_PASS), de forma que no aparecen hardcodeadas en el código subido a GitHub.
- Variables de entorno: la cadena de conexión a la base de datos (DATABASE_URL) se gestiona mediante un archivo .env en local y mediante variables de entorno configuradas en el panel de Render en producción. El archivo .env está incluido en .gitignore para impedir su exposición.
- Middleware de autenticación: todas las rutas de la API requieren sesión autenticada, excepto las rutas públicas explícitamente declaradas (/api/login, /api/me, /login.html).
- SSL en la conexión a la base de datos: la conexión a Neon utiliza SSL (sslmode=require), garantizando el cifrado de los datos en tránsito.

- Control de acceso por rutas: las rutas de administración (crear semana, cerrar semana, añadir jugadores, importar/exportar datos) solo son accesibles para usuarios autenticados.

CE f) Ejercicios sobre gestión de usuarios y políticas de seguridad

Se configuró la política de seguridad de red en Neon: se identificó la opción de restringir las IPs autorizadas a conectarse a la base de datos, evaluando la posibilidad de configurar únicamente la IP de salida de Render para mayor seguridad. Se optó por mantener la configuración permisiva (0.0.0.0/0) durante la fase de desarrollo, reconociendo que en un entorno productivo real se debería restringir al mínimo necesario.

RA03 — Redes virtuales, cómputo, escalado y arquitectura

CE b) Buenas prácticas de seguridad en redes y arquitecturas

La arquitectura diseñada sigue el principio de separación de capas:

- La base de datos (Neon) es un servicio externo independiente al servidor de aplicación (Render), lo que limita la superficie de ataque.
- La aplicación no expone la base de datos directamente: toda la lógica de acceso a datos pasa por el servidor Express, que actúa como capa intermedia.
- Se utilizó HTTPS en producción (gestionado automáticamente por Render como PaaS), garantizando el cifrado de las comunicaciones entre el cliente y el servidor.
- Las credenciales de administración y la connection string nunca se incluyeron en el repositorio de código fuente.

CE c) Creación y configuración de una red funcional

Se configuró la conectividad entre los distintos servicios cloud:

- Servicio web en Render (PaaS): desplegado a partir del repositorio GitHub mediante integración continua. Cada push a la rama principal dispara un redepliegue automático.
- Base de datos en Neon (DBaaS): configurada en la región EU Central para minimizar la latencia desde Europa.
- Conexión entre ambos servicios mediante la variable de entorno DATABASE_URL, con SSL obligatorio.
- Se verificó la conectividad realizando pruebas funcionales de la aplicación una vez desplegada.

CE e) Configuración y gestión de balanceo de carga y escalado automático

Render, como plataforma PaaS, gestiona automáticamente aspectos de escalabilidad que en una infraestructura tradicional requerirían configuración manual:

- Escalado automático de instancias en función de la carga (disponible en planes superiores al gratuito).
- Reinicio automático de la aplicación ante fallos.
- Neon implementa escalado automático de compute: cuando no hay conexiones activas, la base de datos reduce su consumo de CPU a cero (modo sleep), y escala automáticamente al recibir nuevas conexiones.
- Se analizó la diferencia entre este modelo serverless/PaaS y un despliegue tradicional en servidor dedicado, identificando las ventajas en términos de gestión operativa.

CE f) Optimización de recursos computacionales

Se tomaron decisiones de diseño orientadas a la eficiencia:

- Uso de un pool de conexiones (pg.Pool) en lugar de conexiones individuales, reduciendo la sobrecarga de establecer conexiones repetidas con la base de datos.
- Selección del plan mínimo viable en todos los servicios (tier gratuito de Neon, tier gratuito de Render) para el nivel de carga esperado.
- Uso de express.static para servir ficheros estáticos directamente sin procesamiento adicional del servidor.

RA04 — Almacenamiento, bases de datos en la nube, monitorización y optimización

CE b) Configuración y gestión de bases de datos en un entorno de nube

Se realizó la migración completa de la base de datos de SQLite local a PostgreSQL en Neon. Este proceso incluyó:

- Creación del proyecto en Neon y obtención de la connection string.
- Sustitución del driver sqlite3 por pg (node-postgres) en el servidor Express.
- Adaptación de todas las queries: sustitución de los placeholders ? de SQLite por $1, $2, ... de PostgreSQL.
- Sustitución de funciones específicas de SQLite: GROUP_CONCAT → STRING_AGG, AUTOINCREMENT → SERIAL.
- Implementación de la creación automática de tablas al arrancar el servidor (función initDB()), de forma que la base de datos se inicializa correctamente en cualquier entorno sin intervención manual.
- Gestión de secuencias (SERIAL) en las operaciones de importación de datos, reiniciando los contadores tras la restauración de copias de seguridad.
- Verificación del funcionamiento de todas las funcionalidades (CRUD de jugadores, semanas, predicciones, historial, rankings) sobre la base de datos en la nube.

CE c) Resolución de problemas prácticos sobre almacenamiento y bases de datos

Durante el proyecto se identificaron y resolvieron varios problemas técnicos reales:

- Problema de persistencia en Render: al utilizar SQLite local, los datos se perdían en cada redepliegue al ser el sistema de archivos de Render efímero. Solución: migración a base de datos externa (Neon).
- Problema de SSL: la conexión inicial a Neon fallaba por configuración SSL incorrecta. Solución: configurar { ssl: { rejectUnauthorized: false } } en el pool de conexiones.
- Problema de ficheros estáticos: el servidor servía los ficheros estáticos desde el directorio incorrecto, impidiendo la carga del frontend. Solución: verificación y corrección de la ruta en express.static.
- Gestión de credenciales expuestas: al compartir la connection string en un canal no seguro, se procedió a rotar la contraseña desde el panel de Neon como medida de seguridad.

CE e) Uso de herramientas de monitorización y recomendaciones de optimización

Se utilizó el dashboard de Neon para monitorizar el consumo real de recursos:

- Seguimiento de Compute (CU-hrs): se observó el comportamiento del escalado automático, con picos de consumo al iniciar el servidor y consumo nulo durante períodos de inactividad.
- Seguimiento de Storage: se verificó que el almacenamiento crece de forma lineal y controlada con el uso de la aplicación.

- Seguimiento de Network transfer: se comprobó que el tráfico de red es mínimo al tratarse de intercambio de datos JSON de pequeño tamaño.
- Se analizaron las recomendaciones del plan gratuito y se estableció un criterio de alerta: si el consumo mensual superara el 50% del límite, se evaluaría la migración a un plan de pago o la optimización de consultas.

CE f) Análisis y mejora de arquitecturas existentes

Se realizó un análisis crítico de la arquitectura inicial (SQLite local) identificando sus limitaciones y se propuso y ejecutó una arquitectura mejorada:

Arquitectura inicial	Arquitectura mejorada
SQLite en disco local	PostgreSQL en Neon (DBaaS)
Datos volátiles (se pierden al redesplegar)	Datos persistentes independientes del servidor
No escalable	Escalado automático gestionado por Neon
Sin SSL en BD	Conexión SSL obligatoria
Credenciales en código	Variables de entorno + .gitignore
Sin monitorización	Dashboard de Neon con métricas en tiempo real

3. Conclusiones

El desarrollo y despliegue de la aplicación PorraIDS ha constituido un proyecto práctico completo que ha permitido aplicar de forma real los contenidos del módulo Introducción a la Nube Pública. Se han trabajado aspectos de los cuatro resultados de aprendizaje evaluados, desde la comparativa y selección de proveedores cloud hasta la configuración de bases de datos como servicio, pasando por la implementación de medidas de seguridad, el despliegue en plataformas PaaS y la monitorización del consumo de recursos.

La experiencia ha puesto de manifiesto las ventajas concretas de los servicios cloud frente a una arquitectura tradicional: eliminación de la gestión de infraestructura, escalado automático,alta disponibilidad, modelo de pago por uso y facilidad de despliegue mediante integración continua.

4. Imágenes descriptivas de lo realizado

En este apartado voy a mostrar imágenes que reflejan lo anteriormente citado.

RA01 Costes

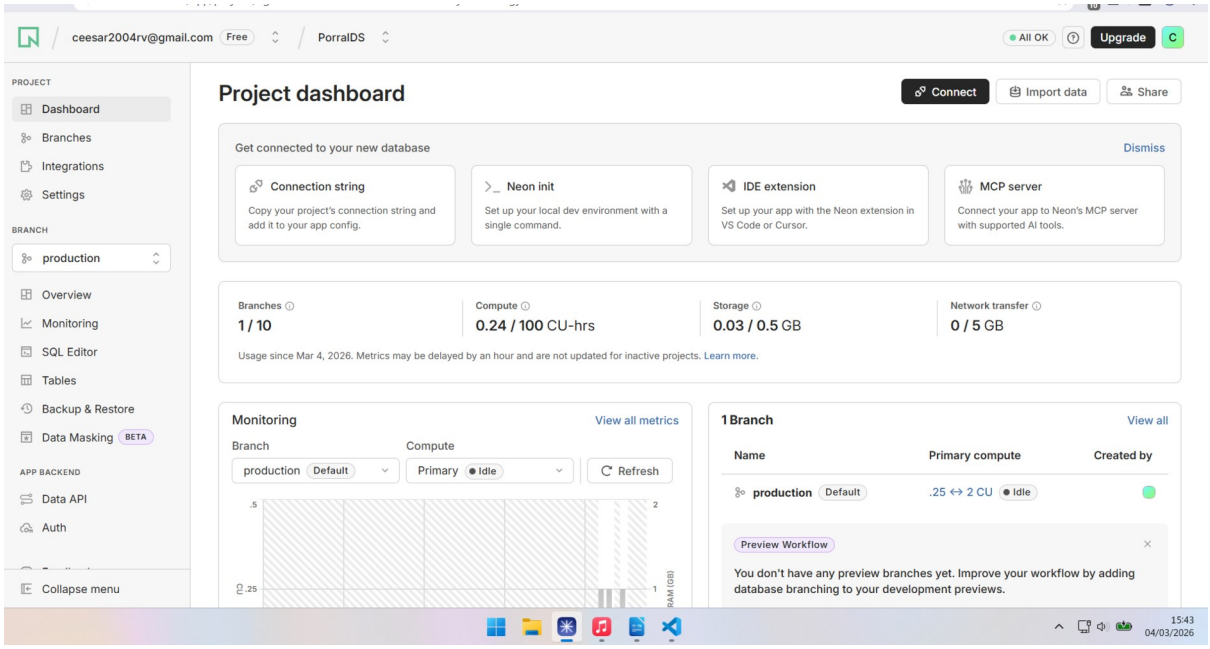

Figura 1: Dashboard de Neon con las métricas de consumo.

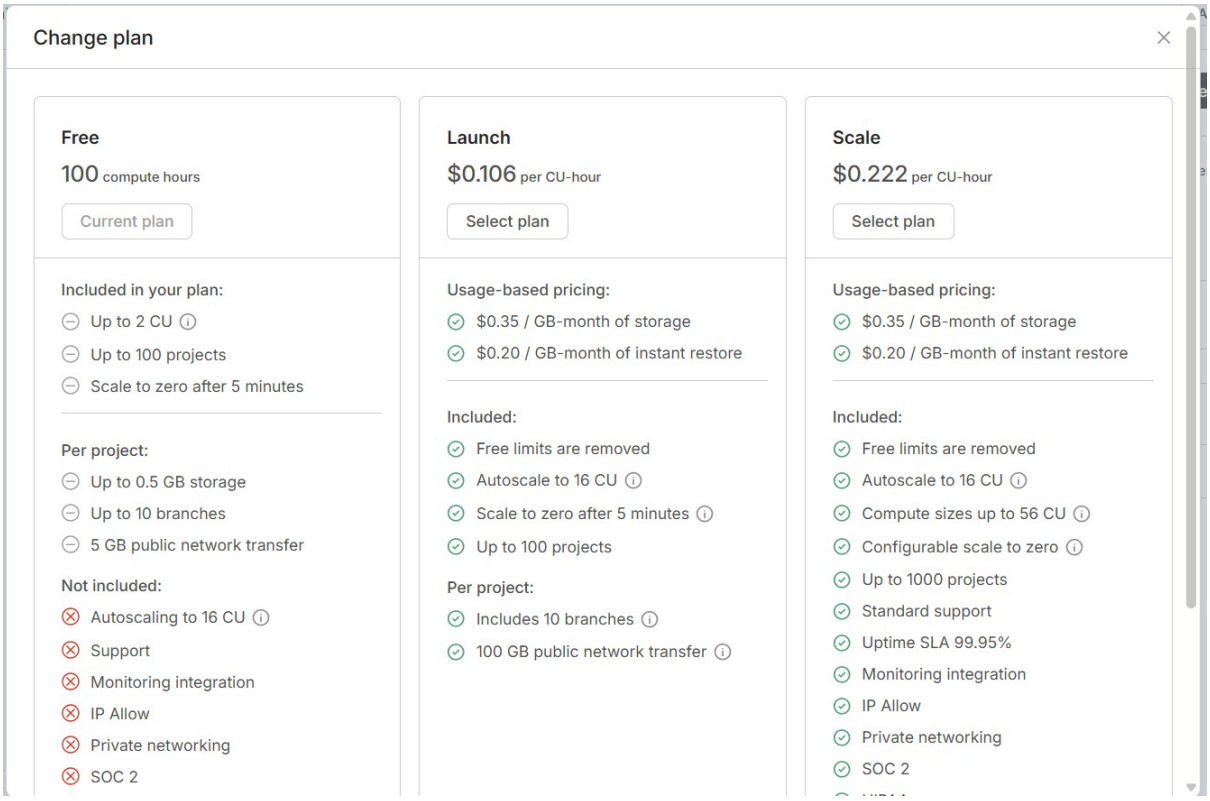

Figura 2: Página de precios de Neon mostrando los diferentes planes.

RA02 Seguridad

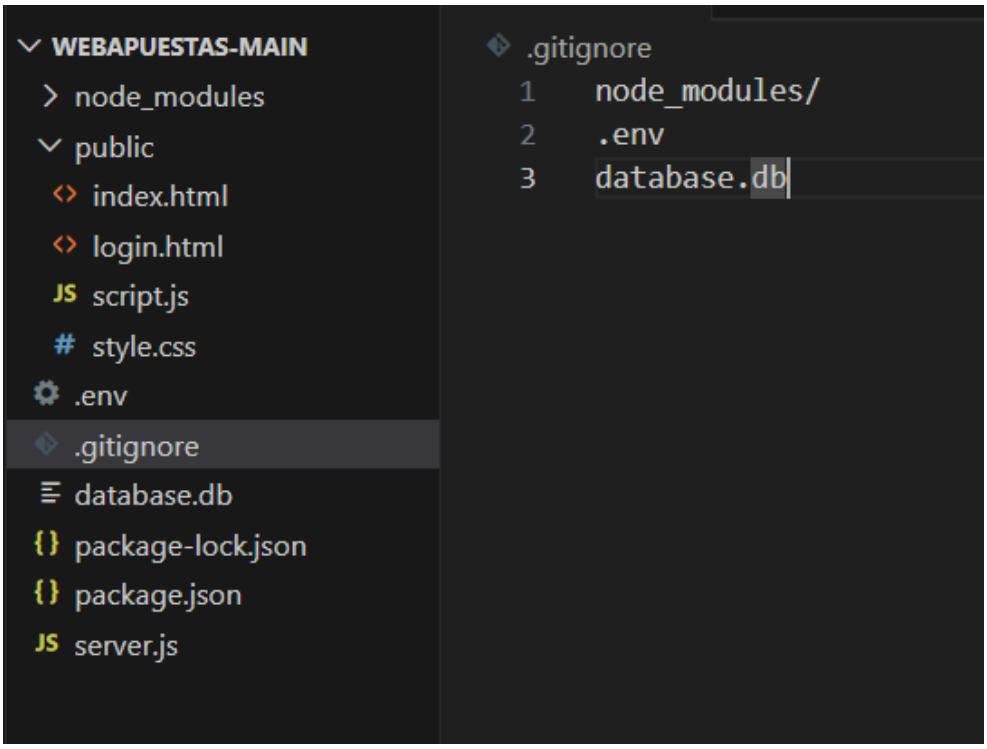

Figura 3: Archivo .gitignore para no subir archivos sensibles a la nube.

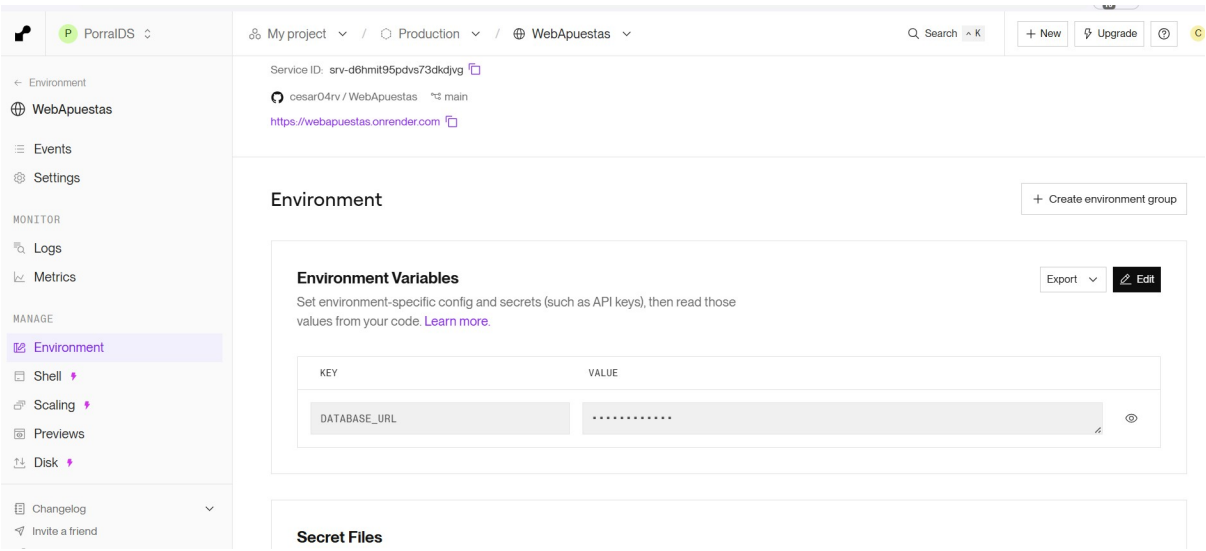

Figura 4: Panel de variables de entorno de Render donde se hace la conexión a la base de datos de Neon.

Figura 5: Aplicación arrancando.

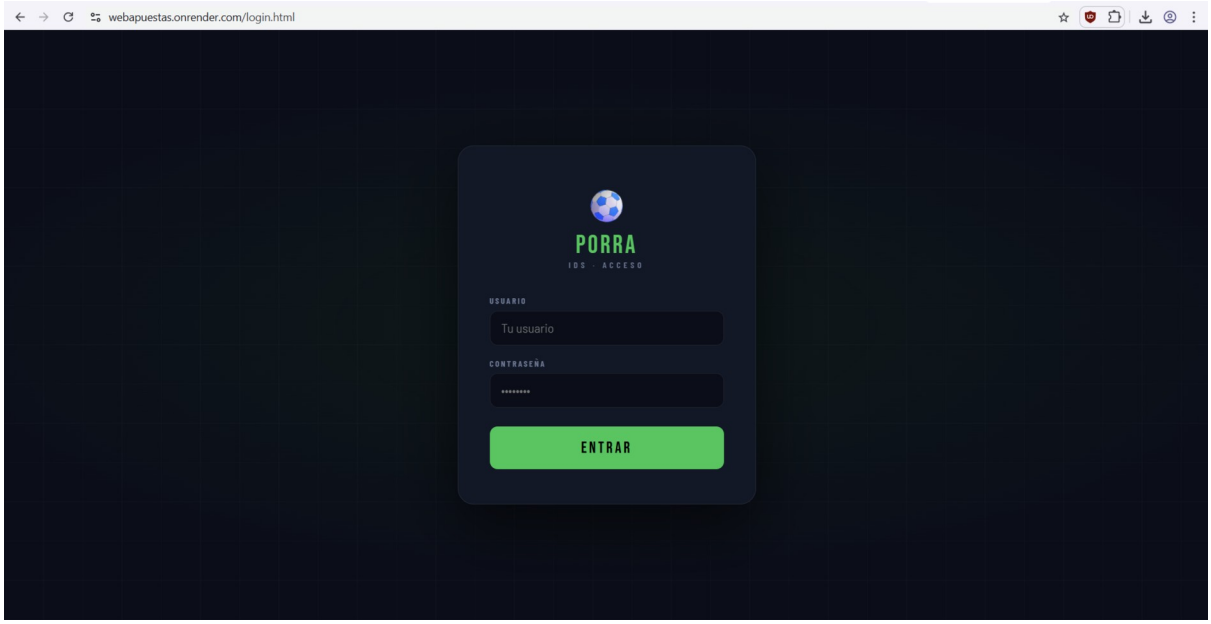

Figura 6: Login de usuarios.

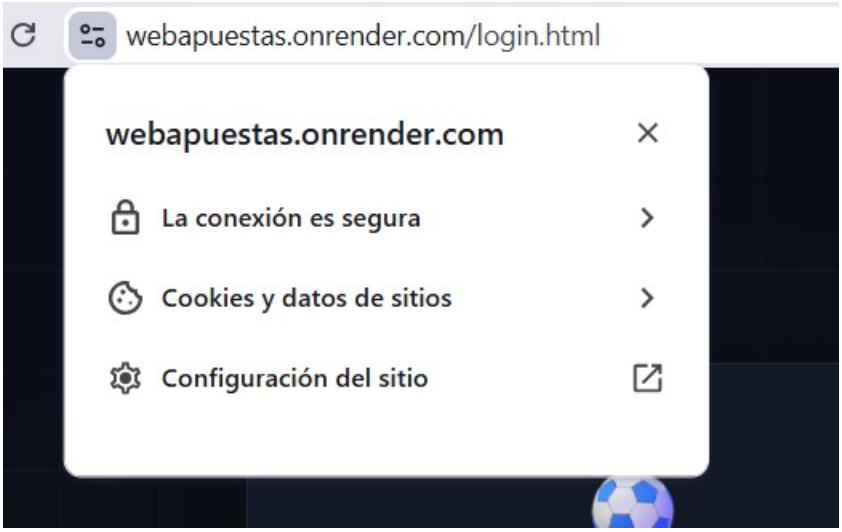

Figura 7: Uso de SSL para mayor seguridad.

RA 03 Redes y arquitectura

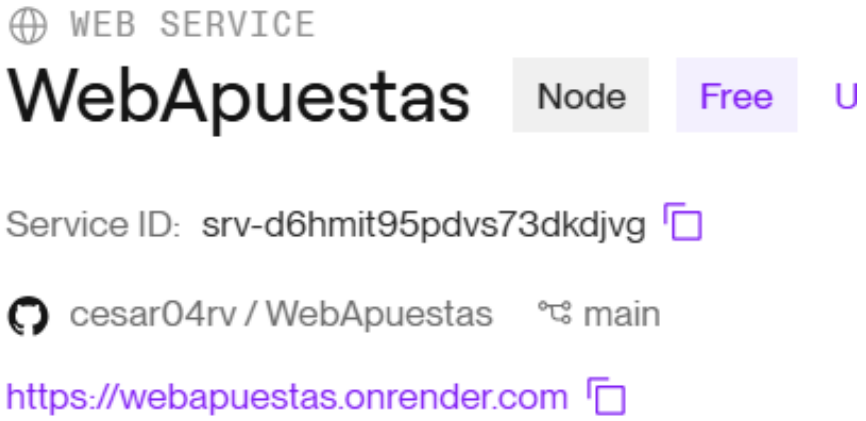

Figura 8: Demostración de conexión con el Github para la actualización de la web.

```
01:23:18 PM  [24h5w]  To prepare for this change:
01:23:18 PM  [24h5w]  - If you want the current behavior, explicitly use 'sslmode=verify-full'
01:23:18 PM  [24h5w]  - If you want libpq compatibility now, use 'uselibpqcompat=true&sslmode=require'
01:23:18 PM  [24h5w]
01:23:18 PM  [24h5w]  See https://www.postgresql.org/docs/current/libpq-ssl.html for libpq SSL mode definitions.
01:23:18 PM  [24h5w]  (Use `node --trace-warnings ...` to show where the warning was created)
01:23:18 PM  [24h5w]  ✅ Base de datos lista
01:23:18 PM  [24h5w]  ✅ Servidor en http://localhost:10000
01:23:18 PM  [24h5w]
01:23:27 PM           ==> Your service is live 🎉
01:23:27 PM           ==>
01:23:27 PM           ==> ///////////////////////////////////////////////////////////
01:23:27 PM           ==>
01:23:27 PM           ==> Available at your primary URL https://webapuestas.onrender.com
01:23:27 PM           ==>
01:23:27 PM           ==> ///////////////////////////////////////////////////////////
01:28:28 PM           ==> Detected service running on port 10000
01:28:29 PM           ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
03:51:38 PM  [xjj91]  ==> Running 'node server.js'
```

Figura 9: pestaña de Deploys de Render donde se ve el historial de despliegues automáticos.

Project settings	Manage
Region	AWS Europe Central 1 (Frankfurt)
Default compute size	.25 ↔ 2 CU
History retention	6 hours
Postgres version	17

To extend the compute size limit, upgrade your plan

Figura 10: Configuración de Neon

RA04 Base de datos

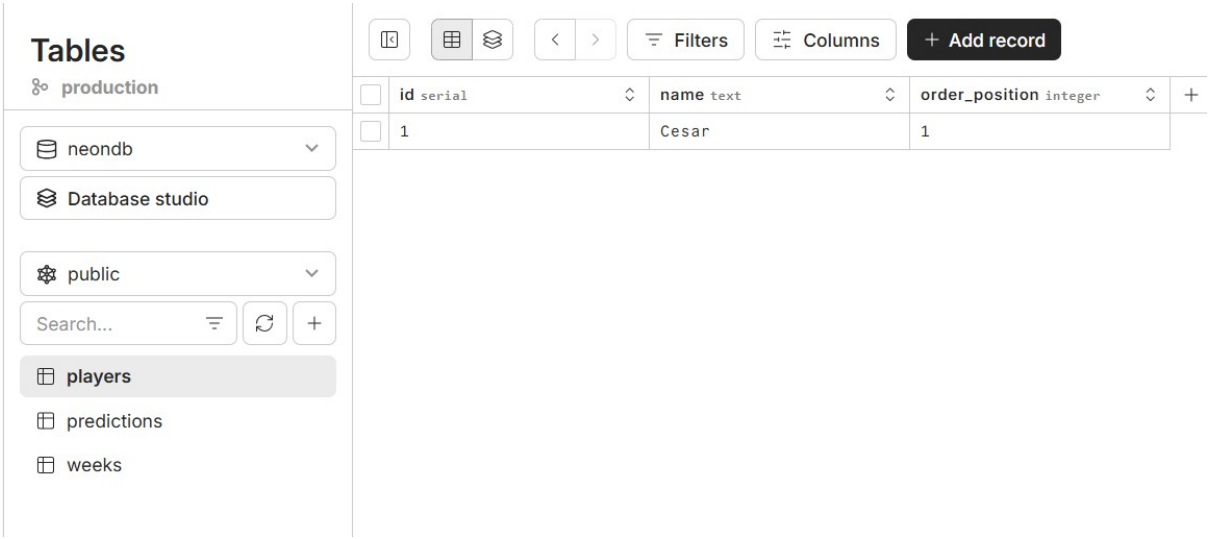

Figura 11: Demostración de las tablas de la base de datos de Neon.

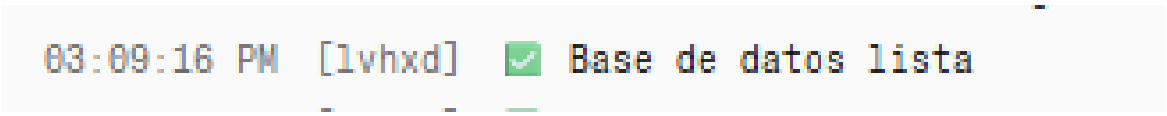

Figura 12: Log de Render, demostrando que la BD funciona.